

Heaters

Company: Amazon

Round: Online Assessment (OA)

Year: 2025

Difficulty: Medium

Topic: Binary Search, Sorting, Arrays, Two Pointers

Source: LeetCode 475 - Heaters

Problem Description

Winter is coming! During the contest, your first job is to design a standard heater with a fixed warm radius to warm all the houses.

Every house can be warmed, as long as the house is within the heater's warm radius range.

Given the positions of houses and heaters on a horizontal line, return the **minimum radius** standard of heaters so that those heaters could cover all houses.

Notice that all the heaters follow your radius standard, and the warm radius will be the same.

Input Format

The first line contains an integer H — the number of houses.

The second line contains H space-separated integers representing the positions of the houses.

The third line contains an integer K — the number of heaters.

The fourth line contains K space-separated integers representing the positions of the heaters.

Output Format

Return a single integer representing the **minimum common radius** required so that every house is covered by at least one heater.

Constraints

- $1 \leq \text{houses.length}, \text{heaters.length} \leq 3 * 10^4$
- $1 \leq \text{houses}[i], \text{heaters}[i] \leq 10^9$
- All heaters use the same radius.
- Every house must be covered by at least one heater.

Sample Input / Output

Example 1

Input:

```
houses = [1,2,3]  
heaters = [2]
```

Output:

1

Explanation:

The only heater was placed at position 2. With radius 1, the heater covers houses at positions 1, 2, and 3.

Example 2

Input:

```
houses = [1,2,3,4]  
heaters = [1,4]
```

Output:

1

Explanation:

The two heaters are placed at positions 1 and 4. A radius of 1 is enough to cover all houses.

Example 3

Input:

```
houses = [1,5]  
heaters = [2]
```

Output:

3

Explanation:

The heater is at position 2. The house at position 5 is 3 units away, so the required radius is 3.

Approach to Solve This Problem:

The key observation is that for every house, we need to find its **closest heater**.

The minimum radius required for that particular house is the distance between the house and its nearest heater.

Therefore:

Answer = maximum of the minimum distances from every house to a heater.

1. Sort the heaters

First, sort the heaters array.

This allows us to efficiently find the heater closest to each house using binary search.

2. Find the closest heater for each house

For every house, use binary search to find the first heater whose position is greater than or equal to the house position.

There are two possible closest heaters:

- The heater immediately to the **right** of the house.
- The heater immediately to the **left** of the house.

Calculate the distance to both and take the smaller one.

3. Update the required radius

For each house:

minimum distance = distance to closest heater

The final answer must cover **every** house, so take the maximum of these minimum distances.

4. Return the maximum distance

That maximum distance is the minimum common radius needed by all heaters.

Java Solution:

```
import java.util.*;
```

```
class Solution {  
    public int findRadius(int[] houses, int[] heaters) {
```

```
        Arrays.sort(heaters);
```

```
        int answer = 0;
```

```
        for (int house : houses) {
```

```
int left = 0;
int right = heaters.length - 1;

// Find the first heater >= house
while (left <= right) {
    int mid = left + (right - left) / 2;

    if (heaters[mid] < house) {
        left = mid + 1;
    } else {
        right = mid - 1;
    }
}

int distanceToRight = Integer.MAX_VALUE;
int distanceToLeft = Integer.MAX_VALUE;

// left is the first heater >= house
if (left < heaters.length) {
    distanceToRight = heaters[left] - house;
}

// right is the last heater < house
if (right >= 0) {
    distanceToLeft = house - heaters[right];
}

int closestDistance =
    Math.min(distanceToLeft, distanceToRight);

answer = Math.max(answer, closestDistance);
}

return answer;
}
```

Alternative Approach: Two Pointers

After sorting both arrays, we can also solve the problem using two pointers.

For each house, move the heater pointer forward while the next heater is closer to the current house.

This avoids performing a binary search for every house.

```
import java.util.*;

class Solution {
    public int findRadius(int[] houses, int[] heaters) {

        Arrays.sort(houses);
        Arrays.sort(heaters);

        int j = 0;
        int answer = 0;

        for (int house : houses) {

            while (j + 1 < heaters.length &&
                Math.abs(heaters[j + 1] - house)
                <= Math.abs(heaters[j] - house)) {
                j++;
            }

            answer = Math.max(
                answer,
                Math.abs(heaters[j] - house)
            );
        }

        return answer;
    }
}
```

Complexity Analysis:

Binary Search Approach

Time Complexity: $O(H \log K + K \log K)$

- Sorting heaters: $O(K \log K)$
- Binary search for each house: $O(H \log K)$

Therefore:

$O(K \log K + H \log K)$

Space Complexity: $O(1)$ auxiliary space, excluding the sorting implementation's internal stack.

Two-Pointer Approach

Time Complexity: $O(H \log H + K \log K)$

- Sort houses: $O(H \log H)$
- Sort heaters: $O(K \log K)$
- Two-pointer traversal: $O(H + K)$

Space Complexity: $O(1)$ auxiliary space, excluding sorting overhead.